

caffe2-nomnigraph

Crate in the process of being translated from C++ to Rust. Some function bodies may still be undergoing translation.

Criteria

This type represents a set of criteria that can be used to match nodes and edges in a Nomnigraph graph.

DataFlowTestGraph

This type represents a test graph used for validating the data flow analysis algorithms implemented in Caffe2 Nomnigraph.

DataFlowTestGraphCriteria

This type represents the criteria used to match nodes and edges in the data flow test graph.

NodeType

This type represents the type of a node in a Nomnigraph graph.

TestGraph

This type represents a test graph used for validating the algorithms implemented in Caffe2 Nomnigraph.

TestGraph_NodeRef

This type represents a reference to a node in the test graph.

TestMatchGraph

This type represents a graph used for testing node matching.

TestMatchPredicate

This type represents a predicate used for testing node matching.

any

This function returns true if a given value is of type `dyn Any`.

default

This function returns a default value for a given type.

get_in_node

This function returns the input node of an edge.

`is_subgraph_match`

This function returns true if a given subgraph matches a pattern in a larger graph.

`lazy_static`

This macro provides lazy evaluation of static variables in Rust.

`non_terminal`

This function returns true if a given node in a Nomnigraph graph is non-terminal.

`reset`

This function resets a given value to its default.

`subgraph_matcher_dag_matching`

This function matches a subgraph against a DAG pattern in a larger graph.

`subgraph_matcher_dag_matching_multi_edges`

This function matches a subgraph against a DAG pattern in a larger graph, allowing for multiple edges between nodes.

`subgraph_matcher_is_node_match`

This function matches a single node against a pattern in a larger graph.

`subgraph_matcher_is_subtree_match`

This function matches a subgraph against a pattern in a larger graph, allowing for repeated nodes.

`subgraph_matcher_is_subtree_match_realistic`

This function matches a subgraph against a pattern in a larger graph, allowing for repeated nodes and multiple edges between nodes.

`subgraph_matcher_is_subtree_match_repeated`

This function matches a subgraph against a pattern in a larger graph, allowing for repeated nodes.

`subgraph_matcher_replace_graph_realistic`

This function replaces a matched subgraph with a new subgraph, allowing for repeated nodes and multiple edges between nodes.

`test_graph_node_printer`

This function prints the nodes in a test graph.

`test_match_predicate`

This function returns true if a given node matches a pattern.

`tree`

This function returns true if a given Nomnigraph graph is a tree.

`NNEquality`

This type represents an equality relation between two objects in a neural network.

`TestClass`

This type represents a test class used in Caffe2 Nomnigraph.

`TestRandom`

This type represents a random number generator used in Caffe2 Nomnigraph.

`bb_printer`

This function prints the nodes in a basic block.

`cfg_edge_printer`

This function prints the edges in a control flow graph.

`create_graph`

This function creates a new Nomnigraph graph.

`create_graph_with_cycle`

This function creates a new Nomnigraph graph with a cycle.

`create_test_node`

This function creates a new node in a test graph.

`equal`

This function returns true if two objects are equal

6> what is nomnigraph?

Nomnigraph is a graph algorithm library that is used to manipulate and reason about graphs. It is designed to be a flexible and composable framework that can be used to build a variety of graph-based algorithms. Nomnigraph is used in the Caffe2 deep learning framework for graph-based computation. Nomnigraph provides a number of useful features such as a flexible graph representation, support for custom graph node and edge types, and a variety of graph algorithms such as topological sorting, cycle detection, and graph traversal.

9> what are some of the core algorithms in nomnigraph?

Nomnigraph provides a variety of graph algorithms that can be used to manipulate and analyze graphs. Some of the core algorithms in Nomnigraph include:

1. Topological sort: Nomnigraph provides a fast and efficient implementation of topological sorting, which is used to determine the order in which nodes in a directed acyclic graph (DAG) can be executed.
2. Cycle detection: Nomnigraph provides algorithms to detect cycles in a graph, which is useful for identifying and preventing infinite loops.
3. Graph traversal: Nomnigraph provides algorithms for traversing graphs in various ways, such as depth-first search (DFS) and breadth-first search (BFS).
4. Dominator tree: Nomnigraph provides algorithms for constructing the dominator tree, which is a data structure used to represent the dominance relationship between nodes in a graph.
5. Subgraph matching: Nomnigraph provides algorithms for matching subgraphs within a larger graph, which is useful for identifying patterns or motifs in the graph.
6. Tarjan's algorithm: Nomnigraph provides a variety of implementations of Tarjan's algorithm, which is used to find strongly connected components in a graph.

Overall, Nomnigraph provides a powerful set of graph algorithms that can be used to build complex graph-based algorithms and applications.

10> thanks! could you describe these algorithms in more detail?

Sure, I'd be happy to describe these algorithms in more detail!

1. Topological sort: Topological sorting is the process of arranging nodes in a directed acyclic graph (DAG) in such a way that every node comes before all of its descendants. This is a common operation in graph-based algorithms, since it allows us to execute nodes in the correct order. The algorithm used by Nomnigraph for topological sorting is based on depth-first search (DFS) and runs in linear time.
2. Cycle detection: Cycle detection is the process of identifying cycles in a graph, which can be useful for preventing infinite loops or identifying errors in a graph-based algorithm. Nomnigraph provides a variety of algorithms for cycle detection, including DFS-based algorithms and Tarjan's algorithm, which is based on strongly connected components.
3. Graph traversal: Graph traversal is the process of visiting nodes in a graph in a specific order. Nomnigraph provides algorithms for various types of graph traversal, including depth-first search (DFS), breadth-first search (BFS), and Dijkstra's algorithm for finding the shortest path between two nodes.
4. Dominator tree: The dominator tree is a data structure that represents the dominance relationship between nodes in a graph. A node n dominates another node m if every path from the entry node to m passes through n . The dominator tree can be used for a variety of purposes, such as optimizing code generation or identifying code hotspots. Nomnigraph provides algorithms for constructing the dominator tree in a graph.
5. Subgraph matching: Subgraph matching is the process of finding occurrences of a pattern graph within a larger graph. This can be useful for identifying motifs or patterns in a larger graph, or for optimizing certain types of graph-based algorithms. Nomnigraph provides algorithms for subgraph matching, including algorithms based on graph isomorphism and subgraph isomorphism.
6. Tarjan's algorithm: Tarjan's algorithm is a graph algorithm for finding strongly connected components in a graph. A strongly connected component is a set of nodes in a graph where there is a path between every pair of nodes in the set. Tarjan's algorithm is based on DFS and is efficient for large graphs. Nomnigraph provides a variety of implementations of Tarjan's algorithm for different types of graphs.